HomeBuild — Data & Response Contract

Every JSON shape the application reads and writes, the complete set of permitted values, and what each of the three PRD personas actually sees.

Two different shapes appear in this document and they are not interchangeable:

Shape	What it is	Where it is defined
Row	A record exactly as the database stores it. snake_case. This is what PostgREST returns and what an RLS policy is evaluated against.	supabase/schema.sql
Response	The UI-shaped object a screen consumes. camelCase, with database vocabulary already mapped to presentation (timeline_status: 'on_track' becomes a green 'On Track' chip).	src/api/*.js

The mapping between them lives in one place per domain — the `src/api/` module — rather than being repeated in every component that renders a chip. A screen never sees a raw row.

> **Read this section first.** The three roles in `profiles.role` are descriptive labels. They do **not** grant or restrict anything: the string 'project_manager' appears in no Row Level Security policy anywhere in the schema. What a person can see is decided entirely by their *relationship to a project*. Section 6 sets this out in full, and it is the part most likely to be assumed wrong.

1. The access model

One function decides every read in the system:

```
has_project_access(project_id) = you are the project's manager_uid
                              OR you are the project's customer_uid
                              OR you appear in project_members for it
```

Everything that hangs off a project — site updates, alerts, milestones, schedule phases, budget lines, cost categories, expenses, floor plans, photos, activity — inherits that check verbatim. There is no per-role branch anywhere in it.

A second function gates administration:

```
is_project_manager(project_id) = you are the project's manager_uid
```

That is what restricts adding and removing project members, and writing cost categories. Again it reads a *column on the project*, not the persona on your profile. A user whose `profiles.role` is 'customer' but who is set as a project's `manager_uid` has full manager rights over it.

What this means in practice

Question	Answer
Does changing profiles.role change what I can see?	No. It changes labels in the UI and nothing else.
Can two personas see the same project?	Yes, and they must — PRD section 4 has a homeowner, a PM and a supervisor all working the same build.
What is actually private?	One project from another. Not one person from another.
Are profiles and contractors scoped?	No, deliberately: both are shared directories. You cannot assign a PM or pick a contractor you cannot see.
Are invitations scoped?	Yes, to the inviter — reading, not just revoking.

Consequence worth knowing before a real launch: any signed-in account can read the full profiles directory, including the email and phone of people they share no project with. This is the one open security decision, and supabase/README.md ranks three fixes for it.

2. Permitted values

Every one of these is enforced by a CHECK constraint. A value outside the set is rejected by the database, not merely discouraged by the client.

Identity and lifecycle

Field	Permitted values
`profiles.role`	customer | project_manager | site_supervisor
`project_members.role`	free text; 'creator', 'manager', 'customer', 'member' are what the app writes
`projects.status`	lead | in_design | active | paused | completed | cancelled
`projects.timeline_status`	on_track | ahead | delayed | at_risk
`projects.budget_status`	on_budget | review_required | over_budget
`projects.plot_facing`	North | South | East | West
`projects.total_floors`	G | G+1 | G+2 | G+3
`projects.bhk_per_floor`	integer 1-4
`projects.progress_pct`	integer 0-100 (derived by trigger; a manual override is possible)
`invitations.status`	pending | accepted | revoked | expired

Workflow states

Field	Permitted values
`milestones.status`	upcoming | pending_verification | approved | paid
`ai_alerts.severity`	Low | Medium | High | Critical (capitalised, unlike every other enum)
`ai_alerts.status`	new | acknowledged | assigned | resolved | false_positive
`ai_alerts.confidence`	numeric 0-1, stored as a fraction and rendered as a percentage
`site_updates.status`	in_progress | done | blocked
`schedule_phases.status`	done | in_progress | critical | delayed | queued | on_hold

Directory and design

Field	Permitted values
`contractors.kind`	internal | sub
`contractors.rating`	numeric 0-5, one decimal
`contractors.score`	integer 0-100
`floor_plans.variant`	A | B | C
`floor_plans.vastu_score`	numeric 0-10
`floor_plans.efficiency_score`	numeric 0-10

Activity log vocabulary

Written by the SQL workflow functions, so this list is exhaustive:

entity	action
project	created | status_changed | progress_updated
milestone	status_changed | second_approved
alert	status_changed
floor_plan	selected
schedule	forecast_applied
account	claimed_demo

Money is stored in rupees, not lakhs. PRD 8.1 says lakhs; the base unit is stored instead so the same figure cannot drift between screens that render it at different magnitudes. Every *_amount, budget_* and cost column is a whole-rupee integer. Formatting to L / Cr happens at the edge.

3. Row shapes — what the database stores

profiles

One row per signed-in user, keyed on the Firebase UID. Firebase owns identity; this table holds what Firebase has nowhere to put.

```
{
  "firebase_uid": "T2mQ...z9", // text PK, from Firebase Auth
  "full_name": "Manish Kumar",
  "email": "manish@example.com",
  "phone": "+91 98450 11223", // null until set
  "job_title": "Senior Project Manager",
  "role": "project_manager", // descriptive only, see section 1
  "avatar_url": "https://.../avatars/T2mQ.../avatar.jpg?v=1757",
  "created_at": "2026-02-11T09:04:22.114Z",
  "updated_at": "2026-09-01T17:20:03.882Z"
}
```

projects — PRD 8.1

```
{
  "id": "11111111-1111-4111-8111-000000000001",
  "name": "Sky Villa 42",
  "client_name": "Global Ventures Inc.", // display label
  "customer_uid": "T2mQ...z9", // FK profiles; null if not linked
  "manager_uid": "K9xd...a1", // FK profiles; grants manager rights
  "status": "active",
  "city": "Bengaluru",
  "plot_address": "42 Whitefield Main Rd, Bengaluru 560066",
  "plot_lat": 12.976750, // numeric(9,6), null until surveyed
  "plot_lng": 77.575210,
  "plot_length_ft": 60,
  "plot_width_ft": 40,
  "plot_facing": "East",
  "total_floors": "G+2",
  "bhk_per_floor": 3,
  "budget_total": 12450000, // rupees
  "budget_consumed": 8466000, // recalculated by trigger from expenses
  "progress_pct": 65, // recalculated by trigger from phases
  "budget_status": "on_budget",
  "timeline_status": "on_track",
  "start_date": "2026-01-15",
  "expected_completion": "2026-09-01",
  "actual_completion": null, // stamped on handover
  "created_at": "2026-01-04T06:12:00.000Z",
  "updated_at": "2026-09-05T11:41:38.201Z"
}
```

PRD 8.1 lists 'Selected Plan' as a field here. It is modelled as floor_plans.is_selected instead, so which plan is chosen has exactly one home and the two sides cannot disagree.

project_members — the access list

```
{
  "project_id": "11111111-1111-4111-8111-000000000001",
  "member_uid": "T2mQ...z9",
  "role": "creator" // creator | manager | customer | member
}
```

milestones — PRD 8.5

```
{
  "id": "44444444-4444-4444-8444-000000000001",
  "project_id": "11111111-...-000000000001",
  "sequence": 1, // unique per project
  "name": "Foundation Complete",
  "contractor_id": "33333333-...-000000000005", // nullable
  "release_pct": 18.00, // derived from amount vs budget
  "release_amount": 2241000, // rupees
  "due_date": "2026-10-12",
  "status": "pending_verification",
  "verification_criteria": "Sub-structure cast, PM site inspection passed",
  "pm_verified_at": null,
  "pm_verified_by": null, // FK profiles - who verified
  "customer_approved_at": null,
  "customer_approved_by": null,
  "second_approval_at": null, // required above INR 5,00,000
  "second_approver_uid": null, // must differ from pm_verified_by
  "paid_at": null,
  "payment_txn_id": null, // "SIM-A1B2C3D4E5F6" once paid
  "cost_category_id": "aaaa...-0001", // which category it is charged to
  "created_at": "2026-01-04T06:12:01.000Z",
  "updated_at": "2026-01-04T06:12:01.000Z"
}
```

ai_alerts — PRD 8.4

```
{
  "id": "77777777-7777-4777-8777-000000000001",
  "project_id": "11111111-...-000000000011",
  "source_update_id": "66666666-...-000000000001", // nullable
  "category": "Rebar spacing mismatch",
  "severity": "Critical",
  "confidence": 0.920, // 0-1; null for a human-logged alert
  "description": "Detected bar spacing of 210mm against a specified 150mm.",
  "location_text": "Vertical columns, Level 4 East",
  "location_in_photo": null, // jsonb bounding box, CV model output
  "status": "new",
  "assigned_to": null, // FK profiles
  "detected_at": "2026-10-24T03:43:00.000Z",
  "resolved_at": null,
  "resolution_photo": null,
  "created_at": "2026-10-24T03:43:00.000Z"
}
```

confidence is deliberately null when a person logs a defect by hand. Putting 1.0 there would make a judgement look like a measurement.

site_updates + site_update_photos — PRD 8.3

```
{
  "id": "66666666-6666-4666-8666-000000000001",
  "project_id": "11111111-...-000000000001",
  "supervisor_uid": "S4pl...m2",
  "work_category": "Slab Casting",
  "location_on_site": "East Wing, Level 4",
  "notes": "Concrete pour underway. Slump tests returned 105mm.",
  "status": "in_progress",
  "milestone_id": null, // optional PRD 8.3 reference
  "workers_present": 24, // optional, PRD 5.2.5
  "materials_used": "40 bags OPC, 2 t steel",
  "client_token": null, // idempotency key for offline retry
  "uploaded_at": "2026-10-24T03:45:00.000Z",
  "created_at": "2026-10-24T03:45:00.000Z"
}

// one row per photo, so each carries its own uploader and caption
{
  "id": "88888888-...-0001",
  "site_update_id": "66666666-...-000000000001",
  "storage_path": "<site_update_id>/<uploader_uid>/k2m9x-a7f3.jpg",
  "caption": null,
  "uploaded_by": "S4pl...m2",
  "created_at": "2026-10-24T03:46:11.000Z"
}
```

schedule_phases — PRD 5.4.2

```
{
  "id": "55555555-...-000000000003",
  "project_id": "11111111-...-000000000001",
  "sequence": 3,
  "name": "Rebar Reinforcement",
  "phase_group": "1. Foundations",
  "baseline_start_week": 2.40, // weeks from the PROJECT's start_date
  "baseline_end_week": 3.40,
  "actual_start_week": 2.50, // null until started
  "actual_end_week": 3.40,
  "forecast_start_week": 3.40, // written by apply_schedule_forecast()
  "forecast_end_week": 4.25,
  "cost": 620000,
  "status": "delayed",
  "delay_days": 4,
  "delay_cause": "Material delay", // PRD 5.4.4
  "delay_cost_impact": 85000,
  "delay_recoverable": true, // null = not yet judged
  "created_at": "2026-01-04T06:12:02.000Z"
}
```

expenses — the spend ledger

```
{
  "id": "eeee...-0001",
  "project_id": "11111111-...-000000000001",
  "category_id": "aaaa...-0001", // FK cost_categories, nullable
  "milestone_id": null, // NON-NULL means this IS an escrow release
  "amount": 2850000,
  "incurred_on": "2026-06-12",
  "vendor": "Sharma Constructions",
  "note": "RMC and steel, ground floor slab",
  "created_by": "K9xd...a1",
  "created_at": "2026-06-12T10:02:00.000Z"
}
```

milestone_id is the load-bearing field here. A row with one is an escrow release, posted by the payment workflow running as definer. The RLS policy forbids a member from inserting, updating or deleting such a row — escrow lines cannot be written or edited from a client, which is what makes the ledger audit-ready. A unique index on it stops a release being billed twice.

Remaining tables, in brief

Table	Shape
floor_plans	{ id, project_id, variant 'A'|'B'|'C', name, vastu_score 0-10, efficiency_score 0-10, total_sqft, cost_per_sqft, total_cost, highlights[], vastu_notes, room_layout{floors[],vastuFindings[]}, validation{checks[],passed}, is_selected, budget_tier, architectural_style, vastu_required, extra_spaces[], parking_spots }
contractors	{ id, code 'CON-8842', name, initials, trade, rating 0-5, score 0-100, kind, score_history int[], compliance_ok }
contractor_assignments	{ contractor_id, project_id }
cost_categories	{ id, project_id, label, pct, tone, sequence, budgeted }
budget_entries	{ id, project_id, period '2026-08-01', amount } - legacy monthly series, now only a fallback for demo projects
invitations	{ id, email, role, project_id, invited_by, status, token, expires_at, accepted_at, created_at }
activity_log	{ id, project_id, actor_uid, entity, entity_id, action, summary, detail jsonb, created_at }

4. Response shapes — what a screen receives

These are the objects returned by `src/api/`. Note the deliberate difference from the rows above: statuses arrive as `{ label, tone }` pairs ready to render, and the raw key is kept alongside so logic can still branch on it.

listProjects() / getProject(id)

```
[{
  "id": "11111111-...-000000000001",
  "name": "Sky Villa 42",
  "client": "Global Ventures Inc.", // "-" when unset
  "city": "Bengaluru",
  "manager": "Manish Kumar", // "Unassigned" when null
  "managerUid": "K9xd...a1",
  "status": "active", // raw lifecycle key
  "progress": 65,
  "budgetTotal": 12450000,
  "budgetConsumed": 8466000,
  "budgetStatus": { "label": "On Budget", "tone": "success" },
  "timeline": { "label": "On Track", "tone": "success" },
  "timelineKey": "on_track",
  "estimate": "2026-09-01",
  "startDate": "2026-01-15",
  "team": ["https://.../a.jpg", "https://.../b.jpg"], // max 3
  "teamNames": ["Manish Kumar", "Rekha Gupta", "Dhawan Singh"],
  "extraTeam": 0 // the "+N" counter
}]
```

tone is one of: success | warning | danger | violet | info | orange | brand | neutral | none. 'none' renders a label with no status dot, which is the mockup's treatment of 'Delayed'.

getDashboardKpis()

```
{
  "activeProjects": 12,
  "delayed": 2,
  "openAlerts": 124,
  "pendingApprovals": 2,
  "budgetHealth": 43 // percent of budget REMAINING, 0-100
}
```

listMilestones(projectId)

```
[{
  "id": "44444444-...-000000000001",
  "milestone": "Foundation Complete",
  "contractor": "SteelWorks Co.", // "Unassigned" when null
  "dueDate": "2026-10-12",
  "amount": 2241000,
  "status": { "label": "Pending Verification", "tone": "warning" },
  "statusKey": "pending_verification",
  "releasePct": 18,
  "criteria": "Sub-structure cast, PM site inspection passed",
  "txnId": null,
  "pmVerifiedAt": null,
  "customerApprovedAt": null,
  "paidAt": null,
  "needsSecondApproval": true, // amount > 500000
  "secondApprovedAt": null,
  "simulated": false // true when txnId starts "SIM-"
}]
```

listAlerts({ severity, status, page })

```
{
  "alerts": [{
    "id": "77777777-...-000000000001",
    "type": "Rebar spacing mismatch", // the category column
    "location": "Vertical columns, Level 4 East",
    "severity": "Critical",
    "confidence": 92, // 0-1 in the row, PERCENT here
    "project": "Grand Sky Plaza",
    "detected": "2026-10-24T03:43:00.000Z",
    "resolved": false,
    "status": "new"
  }],
  "total": 124,
  "totalPages": 13 // ALERTS_PER_PAGE = 10
}
```

listFeedEntries({ projectId, category })

```
[{
  "id": "66666666-...-000000000001",
  "projectId": "11111111-...-000000000001",
  "projectName": "Sky Villa 42",
  "chip": { "label": "SLAB CASTING", "tone": "success" },
  "title": "East Wing, Level 4",
  "category": "Slab Casting",
  "date": "2026-10-24T03:45:00.000Z",
  "time": "09:15 AM",
  "status": { "label": "In Progress", "tone": "success" },
  "note": "Concrete pour underway. Slump tests returned 105mm.",
  "workersPresent": 24,
  "materialsUsed": "40 bags OPC, 2 t steel",
  "team": ["https://.../supervisor.jpg"],
  "supervisor": "Dhawan Singh",
  "photos": [{
    "id": "88888888-...-0001",
    "path": "<update_id>/<uid>/k2m9x-a7f3.jpg",
    "caption": null,
    "url": "https://...signed...&token=eyJ..." // 1 hour TTL
  }],
  "uploadSlots": 2
}]
```

Photo URLs are signed in ONE batch per request, not one call per photo. The site-photos bucket is private; the URL expires after an hour.

getSchedule(projectId)

```
{
  "groups": [{
    "id": "1-foundations",
    "name": "1. Foundations",
    "defaultOpen": true,
    "baseline": { "start": 0, "end": 3.4 },
    "tasks": [{
      "id": "55555555-...-000000000003",
      "name": "Rebar Reinforcement",
      "badge": { "label": "DELAYED", "tone": "danger" },
      "bar": { "start": 2.5, "end": 3.4, "tone": "danger" },
      "forecast": { "start": 3.4, "end": 4.25 }, // null when not forecast
      "flag": "+4 Days", // null when on time
      "cost": 620000,
      "status": "delayed",
      "delayDays": 4,
      "delayCause": "Material delay",
      "delayCostImpact": 85000,
      "delayRecoverable": true
    }]
  }],
  "months": [{ "label": "JANUARY 2026", "weeks": ["W1","W2","W3"] }],
  "today": { "offset": 2.45, "label": "TODAY: OCT 18" }, // or null
  "totalDelayDays": 4,
  "phaseCount": 9
}
```

Week offsets are measured from the PROJECT's start_date, not from a calendar week. today is null when the project has no start date or when today falls outside the twelve-week window, so no marker is drawn on a guess.

getCostCategories(projectId)

```
{
  "uncategorised": 0, // spend naming no category, reported not dropped
  "categories": [{
    "id": "aaaa...-0001",
    "label": "Civil & Structural",
    "value": 48, // planned percentage - drives the donut
    "tone": "brand",
    "budgeted": 5976000,
    "spent": 4090000,
    "usedPct": 68, // UNCAPPED: an overrun reads above 100
    "over": false
  }]
}
```

listTeamMembers(ownerUid) — Settings

Derived from invitations this user sent, not from the profiles directory. Returns `[]` until they invite someone.

```
[{
  "inviteId": "cccc...-0001",
  "id": "T2mQ...z9", // null until they sign up; gates the activity query
  "name": "Rekha Gupta", // the email address when not yet signed up
  "email": "rekha@example.com",
  "role": "Lead Architect", // job_title, else the invited persona label
  "status": "ACTIVE", // ACTIVE | PENDING | EXPIRED
  "joined": true,
  "avatar": "https://.../avatar.jpg",
  "initials": "RG"
}]
```

globalSearch(query) — topbar

```
{
  "term": "harbor",
  "groups": [{
    "key": "projects",
    "labelKey": "search.projects", // translated at render
    "items": [{
      "id": "11111111-...-000000000002",
      "title": "Harbor Heights",
      "detail": "Beacon Properties Ltd. - Mumbai",
      "to": "/projects"
    }]
  }],
  "total": 1
}
```

Five tables are queried in parallel. RLS does the access control, so a search can only ever surface what the caller could already open.

listActivity({ projectId, actorUid })

```
[{
  "id": "dddd...-0001",
  "entity": "milestone",
  "action": "status_changed",
  "summary": "Milestone \u201cFoundation Complete\u201d moved to approved",
  "at": "2026-09-05T11:41:38.201Z",
  "actor": "Manish Kumar", // "System" when actor_uid is null
  "actorAvatar": "https://.../avatar.jpg",
  "project": "Sky Villa 42"
}]
```

5. RPC contracts

These are the writes that must not be bypassed by a client bug, so they live in SQL. Called via `supabase.rpc(name, args)`.

Function	Argument / returns and rules
`create_project`	p_project jsonb (all values as strings; empty string means null). Returns the projects row. Runs as definer so a project can be assigned to a different PM without the creator losing access; always records the creator as a member, and applies the three PRD templates in the same transaction (6 categories, 6 milestones, 11 phases).
`advance_milestone`	(p_milestone_id, p_next_status). Returns the milestone row. Legal moves ONLY: upcoming to pending_verification, pending_verification to approved, approved to paid, pending_verification back to upcoming. Refuses 'paid' above INR 5,00,000 until second_approval_at is set. Stamps timestamps and mints the payment reference server-side.
`second_approve_milestone`	(p_milestone_id). Returns the milestone row. Refuses a milestone that is not 'approved', one below the threshold, and the person who verified it - dual approval one account can satisfy twice is a log entry, not a control.
`resolve_alert`	(p_alert_id, p_next_status, p_assignee). Returns the alert row. p_next_status must be acknowledged | assigned | resolved | false_positive. Writes its own audit entry.
`apply_schedule_forecast`	(p_project_id). Returns integer - how many phases shifted. Applies all three PRD 5.4.3 rules: cumulative delay pushed forward, a 15% monsoon buffer for baseline weeks 22-36, and rework days per OPEN high-severity alert (High adds 3, Critical adds 7).
`select_floor_plan`	(p_plan_id). Returns the floor_plans row. Exclusive - clears the other plans on the project in the same call, and moves a lead/in_design project to active.
`dashboard_kpis`	() Returns a single-row table. Respects RLS, so the counts are of what the caller can see.
`claim_demo_projects`	() Returns integer. Adds the caller to every seeded project, because a correctly empty first sign-in is indistinguishable from a broken one.

6. The three roles — what each actually sees

> The single most important thing on this page: these differences come from **which projects a person is attached to**, not from their `profiles.role`. Two accounts with the same role see completely different data; one account can hold different relationships to two different projects.

How each persona is normally attached

Persona	Typical attachment	What follows from it
customer	projects.customer_uid on their own build	Reads that project and everything under it. Cannot add or remove members. Cannot write cost categories. Can record an expense, raise an alert and post a site update, because those policies ask only for project access.
project_manager	projects.manager_uid, usually on several projects	Everything a customer can do on those projects, plus managing the member list and cost categories, plus deleting the project.
site_supervisor	a project_members row	Reads the project and posts site updates and photos as themselves. No member management. Identical read scope to the customer.

Read that table as the intended arrangement, not as an enforced one. Nothing stops a 'customer' being set as manager_uid, and if that happens they get manager rights.

Write permissions, exactly as the policies read

Operation	Requirement
select, on any project-owned table	has_project_access(project_id)
insert project	manager_uid = you (or go through create_project)
update project	has_project_access(id)
delete project	manager_uid = you
add / remove project member	is_project_manager(project_id); anyone may remove themselves
write cost categories	is_project_manager(project_id)
insert expense	has_project_access AND created_by = you AND milestone_id IS NULL
update / delete expense	created_by = you AND milestone_id IS NULL
insert site update	has_project_access AND supervisor_uid = you
insert site photo	uploaded_by = you AND access to the update's project
insert / update alert	has_project_access(project_id)
insert / update milestone	has_project_access(project_id)
update own profile	firebase_uid = you
read invitations	invited_by = you

The same milestone, seen through the workflow

The payment chain spans all three applications. This app implements the manager's half and *shows* the rest as status rather than as a button:

```
upcoming
  -> pending_verification PM submits (this app)
  -> approved PM verifies (this app)
     [+ second approval if release_amount > 500000, by a DIFFERENT person]
  -> paid CUSTOMER approves (customer app - NOT here)
```

MILESTONE_ACTIONS in `src/api/finance.js` deliberately omits `approved -> paid`. The database still enforces the whole chain, so a manager cannot reach it from here even by accident.

Worked example — one project, three viewers

Project `Sky Villa 42`, with customer_uid = Rajesh, manager_uid = Manish, and Dhawan in project_members.

```
// Manish (manager_uid) - GET /projects
[ { "name": "Sky Villa 42", ... }, { "name": "The Nexus Hub", ... },
  { "name": "Silver Oak Residences", ... } ] // all he manages

// Rajesh (customer_uid) - GET /projects
[ { "name": "Sky Villa 42", ... } ] // only his own build

// Dhawan (project_members) - GET /projects
[ { "name": "Sky Villa 42", ... }, { "name": "Oak Creek Estates", ... } ]

// Priya, no relationship to any of them - GET /projects
[ ] // not an error; empty
```

An empty array is the correct answer for someone with no project relationship, and it is indistinguishable from a broken app. That is why the Projects screen offers 'Join the demo projects' (`claim_demo_projects()`) rather than leaving a new account staring at nothing.

7. Error shapes

PostgREST errors are translated once, in `src/api/client.js`, so every screen handles them the same way. The client throws an Error carrying a message written for a person:

Code	Rendered message
42P01	The database tables are missing. Run supabase/schema.sql, then seed.sql.
42501	You do not have permission to read this. Check that Firebase is registered as a third-party auth provider in Supabase.
23505	Context-dependent: 'That person is already on this project.' or 'There is already a pending invitation for that address.'
network	Could not reach the database. Check your connection.
other	Passed through with its code appended, so a bug report has something to go on.

RPC functions raise plain exceptions, which surface as the message text — for example *'Cannot move a milestone from upcoming to paid'* or *'The second approval must come from someone other than the verifier'*.

Not implemented, and honest about it

Area	State
AI floor plan generation	The engine, validators and Vastu scoring are real and deterministic. The model call is a documented seam in src/lib/planGenerator.js, not a stub that pretends.
Computer-vision quality checks	No model. Alerts come from the seed or from a person logging one.
Escrow payments and KYC	No provider connected. payment_txn_id is prefixed 'SIM-' and every receipt says so on its face.
Email delivery of invitations	The invite is recorded, expires and can be revoked. Nothing is sent.
Push notifications, direct messaging	Absent. The topbar message icon says so.

Generated by scripts/build_api_contract.py from supabase/schema.sql and src/api/. Regenerate after changing either; where this document and the code disagree, the code is right.